



INSTITUT
POLYTECHNIQUE
DE PARIS

RAPPORT DE TP

Parallélisation d'un code de simulation de galaxie

Travail réalisé dans le cadre de l'UE
Systèmes Parallèles et distribués - OS202

Par
NOUKOUA MAEVA

Soumis à l'appréciation de
M. PARRET-FREAUD AUGUSTIN

Mars 2026
Palaiseau, France

Table des matières

Table des matières	i
Introduction	i
1 Question préliminaire	ii
2 Comparaison entre calcul et affichage	iii
2.1 Conclusion	iii
3 Parallélisation du calcul avec numba	iv
3.1 Résultats expérimentaux	iv
3.2 Analyse des performances	iv
3.3 Conclusion	iv
4 Séparation de l’affichage et du calcul	vi
4.1 Principe	vi
4.2 Résultats	vi
4.3 Analyse	vi
4.4 Conclusion	vi
5 Parallélisation du calcul avec MPI	vii
5.1 Résultats expérimentaux	vii
5.2 Analyse	vii
5.3 Discussion	vii
5.4 Conclusion	vii
6 Pour aller plus loin : Barnes–Hut	viii
6.1 Distribution des boîtes	viii
6.2 Parallélisation du calcul	viii
6.3 Limites	viii
6.4 Conclusion	viii

Introduction

Nous étudions les performances d'une simulation gravitationnelle de type N -corps représentant une galaxie en rotation autour d'un trou noir central.

Le travail demandé consiste à identifier les goulots d'étranglement du code, à mesurer précisément les temps d'exécution, puis à proposer plusieurs stratégies de parallélisation : parallélisation locale via `numba`, séparation calcul/affichage avec MPI, puis distribution du calcul entre processus.

Dans un premier temps, nous analysons le comportement du code de référence, puis nous étudions successivement l'impact des optimisations proposées.

1 | Question préliminaire

En observant la forme de la galaxie simulée, on remarque que sa dynamique est principalement contenue dans le plan (x, y) . L'épaisseur suivant l'axe z reste faible devant l'extension radiale.

Il n'est donc pas intéressant de choisir une valeur de N_k supérieure à 1, car cela reviendrait à raffiner la discrétisation selon une direction où la structure varie peu. Un tel choix augmenterait le coût mémoire et le coût de calcul sans apporter de gain significatif en précision.

On prend donc naturellement $N_k = 1$.

2 | Comparaison entre calcul et affichage

Afin d'identifier le principal goulot d'étranglement du programme, nous avons comparé le temps de calcul pur au temps total de la simulation incluant l'affichage graphique.

Les mesures ont été réalisées sur le jeu de données `galaxy_1000`, avec un pas de temps $\Delta t = 0.0015$, une grille (15, 15, 1) et 20 itérations. Trois exécutions indépendantes ont été effectuées.

TABLE 1 – Comparaison calcul / affichage

Run	Calcul (ms)	Total (ms)	Affichage (ms)
1	88.33	194.74	106.41
2	89.95	170.74	80.79
3	91.34	172.04	80.70
Moyenne	89.87	179.17	89.30

Le coût de l'affichage a été estimé par la relation :

$$T_{\text{affichage}} \approx T_{\text{total}} - T_{\text{calcul}}$$

On observe que le temps de calcul et le temps d'affichage sont du même ordre de grandeur, chacun représentant environ 50% du temps total.

Par ailleurs, on constate que le temps de calcul est plus élevé lorsque l'affichage est activé (environ 90 ms contre 65 ms sans affichage). Cela s'explique par une contention des ressources matérielles (CPU/GPU) ainsi que par des effets de synchronisation liés au rendu graphique.

2.1 Conclusion

Ces résultats montrent que le calcul des interactions gravitationnelles représente une part significative du temps d'exécution, au même titre que l'affichage.

Cependant, le calcul étant la partie algorithmique principale et la plus coûteuse en complexité, il constitue la cible prioritaire pour les optimisations et la parallélisation dans la suite du TP.

3 | Parallélisation du calcul avec numba

3.1 Résultats expérimentaux

Les performances de la version parallélisée ont été évaluées en faisant varier le nombre de threads. Les mesures ont été réalisées sur le jeu de données `galaxy_1000` avec 20 itérations.

TABLE 2 – Temps de calcul pour `galaxy_1000`

Threads	Temps (ms)	Speedup	Efficacité
1	6.95	1.00	1.00
2	3.68	1.89	0.94
4	2.16	3.22	0.80
8	1.82	3.82	0.48

TABLE 3 – Temps de calcul pour `galaxy_5000`

Threads	Temps (ms)	Speedup	Efficacité
1	88.97	1.00	1.00
2	46.51	1.91	0.96
4	26.06	3.41	0.85
8	21.59	4.12	0.52

3.2 Analyse des performances

Les résultats montrent une nette amélioration des performances grâce à la parallélisation. Le temps de calcul diminue significativement lorsque le nombre de threads augmente.

Pour les deux jeux de données, le speedup est proche de linéaire jusqu'à 4 threads, avec une efficacité supérieure à 80%. Cela indique que la parallélisation est particulièrement efficace dans cette zone.

Au-delà de 4 threads, on observe une diminution de l'efficacité. Cela s'explique par plusieurs facteurs :

- la surcharge liée à la gestion des threads ;
- la contention mémoire entre les cœurs ;
- la limite du parallélisme disponible dans le problème.

Le phénomène est plus marqué pour le cas `galaxy_1000`, où la taille du problème est plus faible, ce qui limite le gain apporté par un grand nombre de threads.

En revanche, pour `galaxy_5000`, le speedup reste élevé même avec 8 threads, ce qui montre que la parallélisation est d'autant plus efficace que la taille du problème augmente.

3.3 Conclusion

La parallélisation avec numba permet d'obtenir un gain de performance significatif, avec un speedup pouvant dépasser un facteur 4 sur les cas les plus lourds.

La boucle sur les corps constitue une source de parallélisme naturelle, car les calculs associés à chaque particule sont indépendants. Cela explique l'efficacité de la stratégie adoptée.

Cependant, les gains ne sont pas parfaitement linéaires, en raison des limitations matérielles et des coûts de synchronisation.

Cette optimisation constitue néanmoins une amélioration majeure du code, réduisant drastiquement le temps de simulation et rendant possible le traitement de systèmes de grande taille.

4 | Séparation de l’affichage et du calcul

4.1 Principe

Dans cette version, le calcul et l’affichage sont séparés à l’aide de MPI. Le processus de rang 0 est chargé de l’affichage, tandis que le processus de rang 1 effectue la mise à jour des trajectoires puis envoie les positions au processus d’affichage.

4.2 Résultats

Les temps de calcul mesurés sur le processus de rang 1 sont les suivants :

Threads numba	Temps moyen calcul (ms)
1	53.25
2	51.49
4	49.35

À titre de comparaison, la version numba seule donnait :

Threads numba	Temps moyen calcul (ms)
1	6.95
2	3.68
4	2.16

4.3 Analyse

On constate que la séparation de l’affichage et du calcul n’améliore pas les performances du calcul. Au contraire, les temps mesurés sont beaucoup plus élevés que dans la version numba seule et varient peu avec le nombre de threads.

Cela s’explique par plusieurs facteurs :

- le coût des communications MPI entre les deux processus ;
- la synchronisation entre calcul et affichage ;
- le bruit introduit par l’environnement graphique, qui perturbe les mesures.

Les valeurs maximales très élevées observées pendant les mesures montrent que cette version n’est pas adaptée à une évaluation précise du temps de calcul.

4.4 Conclusion

La séparation affichage/calcul avec MPI ne permet pas ici d’obtenir une accélération intéressante. La version numba seule reste nettement plus efficace pour paralléliser le calcul des trajectoires.

Cette approche peut améliorer l’organisation du programme, mais elle n’est pas le bon choix pour optimiser les performances du calcul lui-même.

5 | Parallélisation du calcul avec MPI

5.1 Résultats expérimentaux

Les performances ont été mesurées pour différents nombres de processus MPI et de threads numba.

Processus MPI	Threads numba	Temps moyen (ms)
1	1	6.98
2	1	4.00
2	2	2.30
2	4	2.23

5.2 Analyse

L'utilisation de deux processus MPI permet déjà de réduire significativement le temps de calcul, passant de 6.98 ms à environ 4.00 ms, soit une accélération d'environ 1.75.

L'ajout de parallélisme numba à l'intérieur de chaque processus améliore encore les performances. Avec 2 threads, le temps descend à 2.30 ms, puis à 2.23 ms avec 4 threads.

On observe cependant que le gain devient marginal entre 2 et 4 threads, ce qui indique une saturation des ressources matérielles (nombre de cœurs limité et coût de synchronisation).

5.3 Discussion

Cette approche hybride MPI + numba permet d'exploiter à la fois le parallélisme inter-processus et intra-processus.

Toutefois, les performances restent limitées par :

- le coût des communications MPI nécessaires pour synchroniser les données ;
- le déséquilibre de charge dû à la distribution non uniforme des étoiles ;
- les limitations matérielles (nombre de cœurs disponibles).

5.4 Conclusion

La parallélisation MPI du calcul combinée à numba permet une accélération significative du code.

Néanmoins, les gains ne sont pas parfaitement linéaires en raison des coûts de communication et du déséquilibre de charge, ce qui met en évidence les limites des approches parallèles sur des données non uniformément distribuées.

6 | Pour aller plus loin : Barnes–Hut

6.1 Distribution des boîtes

Une manière naturelle de paralléliser consiste à répartir les sous-boîtes du quadtree entre les processus MPI. Chaque processus s'occupe d'un ensemble de régions de l'espace ainsi que des étoiles qu'elles contiennent.

Les niveaux supérieurs de l'arbre peuvent être connus de tous les processus, car ils sont peu nombreux et utiles pour les approximations globales, tandis que les sous-boîtes plus profondes sont distribuées.

6.2 Parallélisation du calcul

Chaque processus calcule l'accélération des étoiles dont il est responsable.

Pour les interactions proches, le calcul est fait directement avec les étoiles locales. Pour les interactions lointaines, on utilise les centres de masse des boîtes, comme dans l'algorithme Barnes–Hut.

Des échanges MPI permettent de partager les informations nécessaires entre les processus.

6.3 Limites

La principale difficulté vient du fait que la distribution des étoiles n'est pas uniforme : les régions proches du centre contiennent beaucoup plus d'étoiles.

Cela peut entraîner un déséquilibre de charge entre processus. De plus, la structure en arbre rend les communications plus complexes que dans le cas d'une grille régulière.

6.4 Conclusion

Cette approche est plus efficace pour de grandes simulations, mais elle est plus difficile à paralléliser efficacement en MPI.